

GOLLIS UNIVERSITY

HARGAISA TEXT

Hargeisa Municipal Tax & Property Management System

SOFTWARE REQUIREMENTS SPECIFICATION

Document Detail	Information
Prepared by	Ridwaan Cabdirahman Maxamed
Institution	Gollis University
Document version	1.0
Date	August 7, 2026
Status	University Project Documentation

This specification describes only the behavior confirmed in the supplied source code, configuration, database schema, tests, and user interface.

Table of Contents and Document Information

Section	Content	Page
1	Cover Page	1
2	Table of Contents and Document Information	2
3	Introduction	3
4	Overall System Description	4
5	Functional Requirements	5
6	User Roles, Use Cases, and User Flow	6
7	Data and Database Requirements	7
8	External Interface and UI Requirements	8
9	Non-Functional Requirements	9
10	Constraints, Testing, Conclusion, and Future Enhancements	10

2.1 Document Control

Item	Value
Title	Hargaisa Text – Software Requirements Specification
Version / date	1.0 / August 7, 2026
Author / institution	Ridwaan Cabdirahman Maxamed / Gollis University
Source basis	Repository code, README, SQL schema, tests, configuration, UI pages, and existing documentation

2.2 Purpose and Audience

This document defines the verified software requirements of the Hargeisa municipal property portal. It provides a shared reference for academic supervisors, students, municipal administrators, software developers, testers, deployment staff, and future maintainers.

Interpretation rule: anything not confirmed by the current implementation is labelled “Not specified in the current project” or “Future Enhancement.”

3. Introduction

3.1 Background and Problem Statement

Municipal property information is difficult to manage when ownership contacts, listings, maps, status changes, reports, and administrative records are kept in separate files. The implemented system centralizes these activities in a browser-based portal for authorized personnel. It gives staff one controlled source for property and client records, operational reporting, and audit evidence.

3.2 Purpose and Objectives

The project purpose is to support consistent property administration for Hargeisa. Its verified objectives are to:

- authenticate personnel and enforce role-based access;
- create, search, filter, update, map, export, and where authorized delete property listings;
- maintain client records and link clients to properties;
- present dashboards, counts, trends, realized revenue, and a property-location map;
- produce analytics exports and an illustrative per-property tax bill;
- record activity, manage users, notifications, preferences, and database backups.

3.3 Scope

The scope includes a responsive React web client, an Express REST API, MySQL/MariaDB persistence, uploaded property/avatar images, OpenStreetMap display through Leaflet, and administrator-controlled operational tools. The system is an internal administrative portal rather than a public citizen service.

3.4 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
GIS	Geographic Information System
JWT / CSRF	Signed session token / cross-site request-forgery protection
Property listing	A rent or sale record with status, price, location, optional client, coordinates, and images
Assessed value	In this implementation, the sum of listing prices; it is not a formal valuation workflow

4. Overall System Description

4.1 System and Product Perspective

Hargaisa Text is a three-tier full-stack application. A React 19 and TypeScript client runs in the browser; an Express 4 API applies validation, security, and business rules; and MySQL/MariaDB stores operational data. Vite proxies /api and /uploads during development. Docker Compose can deploy client, server, and MariaDB containers.

Layer	Implemented responsibility
Presentation	Responsive pages, tables, forms, charts, map, theme/language preferences, and protected navigation
Application/API	Authentication, RBAC, validation, CRUD operations, analytics, exports, audit, uploads, and backup/restore
Data	Relational records, foreign keys, indexes, JSON audit snapshots, and shared rate-limit buckets

4.2 Main Modules

Implemented modules are: sign-in/session; regional dashboard; property inventory and listings; clients; property analytics; tax management; GIS map; reports/exports; notifications; profile and settings; user management; activity logs; and database backup/restore.

4.3 User Categories

Role	System access
Administrator	All modules; user administration; analytics/tax pages; audit logs; backups; destructive record actions
Agent	View all shared operational pages; create and edit properties/clients and manage property images; no admin-only modules or delete
Viewer	Read-only access to shared dashboards, property/client information, GIS, reports, profile, and settings

4.4 Operating Environment, Assumptions, and Dependencies

- Modern browser; Node.js 20.19+ or 22.12+; MySQL-compatible server; network access between browser, API, database, and map tiles.
- Uploaded files and backups require writable persistent storage; mysql and mysqldump are required for restore/backup.
- Production requires HTTPS and secure secrets. Municipal tax law, formal assessment rules, SLA, hardware baseline, and data-retention policy are not specified in the current project.

5. Functional Requirements

The following grouped requirements cover the observable functions found across the UI, routes, controllers, services, schema, and tests. “Input → process → output” is summarized to keep each requirement testable.

ID	Name	Actor	Input	Process	Expected output
FR-01	Authenticate	All	Username, password, remember option	Validate credentials; rate-limit; issue JWT cookie	Authorized session or clear error
FR-02	Dashboard	All	Authenticated request	Aggregate counts, types, statuses, trends, sold-price revenue	Overview cards/charts/listings
FR-03	Properties	All / Agt+	Search, filters, fields	List/read; admin/agent create or edit; admin delete	Paged validated property records
FR-04	Images	Admin, Agent	JPG/PNG/WebP files	Validate size/type; attach or remove from property	Stored image URLs/gallery
FR-05	Clients	All / Agt+	Contact data, search/page	List/read; admin/agent create/edit; admin delete	Client record linked to property
FR-06	GIS	All	Latitude/longitude	Plot eligible properties on Leaflet map	Interactive mapped markers
FR-07	Reports	All; admin analytics	Filters and export choice	Generate CSV/Excel; admin analytics PDF	Downloadable report file
FR-08	Tax bill	Admin	Property identifier	Apply flat illustrative 1% to listing price	Per-property PDF bill
FR-09	Personal tools	All	Profile, password, avatar, preferences	Validate and persist changes; list/read notifications	Updated account/settings/read state
FR-10	Users/RBAC	Admin	User details, role/status/password	Create and administer accounts; enforce protected routes	Controlled admin/agent/viewer access
FR-11	Audit	Admin	Date/page filters	Record key changes with actor, IP, before/after JSON	Searchable activity log
FR-12	Backup	Admin	Create/download/SQL upload	Run mysqldump or mysql after filename/file checks	Backup file, download, or restore result

Authorization shorthand: “Agt+” means Administrator or Agent. All record-changing requests are validated by the API.

6. User Roles, Use Cases, and User Flow

6.1 Role Rules

Administrators control sensitive modules and deletions. Agents maintain operational property and client data. Viewers consume shared information without changing it. The client hides restricted navigation, while the API independently rejects unauthorized requests.

6.2 Representative Use Cases

Use case	Actor	Preconditions	Main steps	Expected result
UC-01 Sign in	Any user	Active account exists	Enter credentials → validation → session cookie	Role-appropriate workspace opens
UC-02 Maintain listing	Admin/Agent	Signed in; valid data	Open listings → add/edit → optionally upload images	Validated property is saved and logged
UC-03 Review map/report	Any user	Signed in	Choose GIS or Reports → filter/view/export	Map or CSV/Excel output is shown
UC-04 Administer system	Admin	Admin session	Manage users, review logs, create/download/restore backup	Administrative action completes or fails safely
UC-05 Tax document	Admin	Property exists	Open tax page → filter → generate bill	Illustrative 1% PDF bill downloads

6.3 Main User Flow

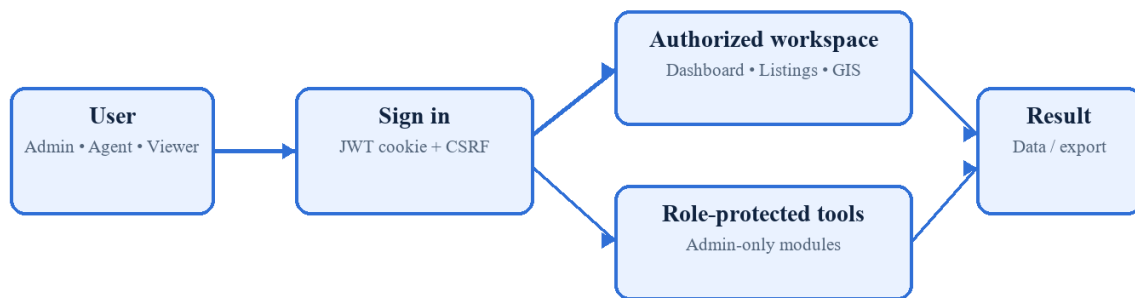


Figure 1. Authentication precedes shared or role-protected work; results are displayed or exported.

7. Data and Database Requirements

7.1 Storage Model

The API uses MySQL/MariaDB with InnoDB and utf8mb4. Keys, foreign-key actions, indexes, ENUM constraints, timestamps, and server-side validation protect the data model. Passwords are stored as bcrypt hashes, never as plain text.

Entity/table	Important stored information
users	Accounts, role, active state, creator, avatar reference
clients	Owner/contact information and optional notes
properties	Listing details, price, location, coordinates, type/status, client and creator
property_images	Many image URLs for one property; cascade delete
notifications / reads	Events plus per-user read status
user_settings	One preference record per user
activity_logs	Actor, action, entity, IP, description, old/new JSON
rate_limit_buckets	Shared request-window counters

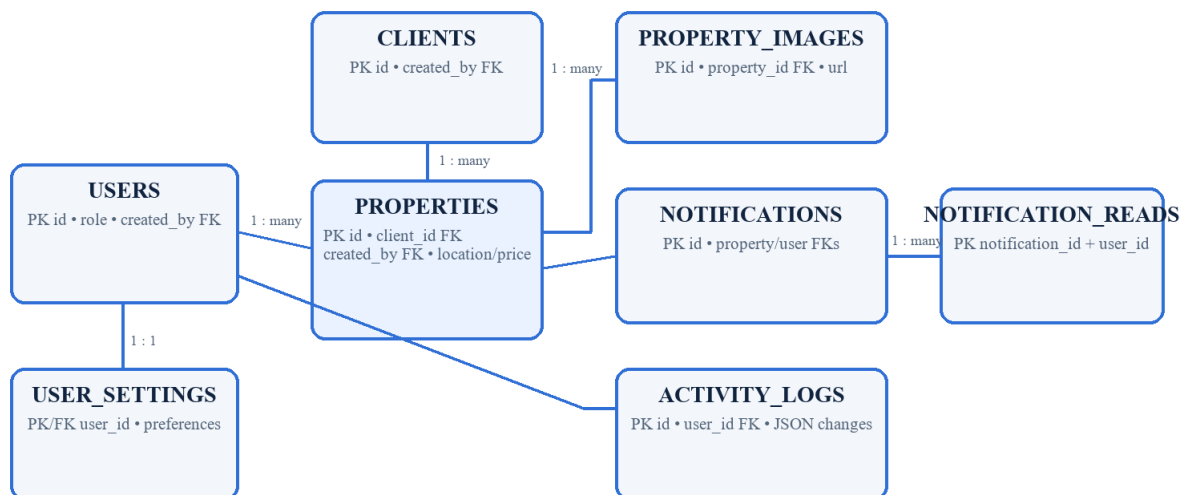


Figure 2. Simplified ERD; properties connect users, clients, images, and notifications. Supporting user data connects settings, reads, and logs.

Key validation: positive prices; latitude $-90...90$; longitude $-180...180$; type rent/sale; status available/sold/rented; role admin/agent/viewer; unique username and email.

8. External Interface and UI Requirements

8.1 Main Screens and Navigation

Screen group	Interface requirement
Sign In	Credentials, show/hide password, remember device, validation feedback
Dashboard / Inventory	Regional cards, counts, composition, trend, property-image listings
Property Listings / Clients	Paged tables, search, filters, CRUD forms, image management, exports
Analytics / Tax	Admin charts and PDF analytics; illustrative property-tax roll and bill
GIS / Reports	Leaflet property markers; filtered CSV and full Excel reports
Profile / Settings	Identity, password, avatar; theme, language, timezone, date and notification choices
Admin screens	User management, activity log, and backup/restore

8.2 Software and API Interfaces

- Browser ↔ Express JSON REST API under /api; uploaded files under /uploads; Swagger UI at /api/docs and OpenAPI JSON at /api/docs.json.
- Express ↔ MySQL/MariaDB through mysql2; mysql/mysqldump command-line tools for restore/backup; PDFKit and ExcelJS for documents.
- Leaflet/react-leaflet displays OpenStreetMap tiles. No payment gateway or other production third-party business service is implemented.

8.3 Responsive, Accessibility, and Hardware

The layout includes desktop navigation and a mobile drawer; tables/forms should remain operable on common modern browsers and screen sizes. Existing controls use labels, button text, icons, keyboard-focusable elements, status text, and feedback messages. Formal WCAG conformance testing is not specified in the current project. Dedicated hardware interfaces are not required; a normal client computer, server, storage, and network connection are assumed.

8.4 Navigation Rule

Unauthenticated user → Sign In. Authenticated user → shared workspace. Administrator → additional analytics, tax, activity, user, and backup pages. Unknown routes → Sign In.

9. Non-Functional Requirements

Where the code provides no numeric service target, the requirement below states the observable control and records the missing threshold explicitly.

ID	Quality	Verified requirement
NFR-01	Performance	Lists shall be paged (limit 1–100); export processing is capped. Response-time target: not specified.
NFR-02	Security	Use bcrypt, signed httpOnly JWT cookies, CSRF tokens, CORS, Helmet, validation, sanitization, RBAC, and rate limiting.
NFR-03	Privacy	Restrict data to authenticated municipal personnel and avoid exposing password hashes. Retention/privacy policy: not specified.
NFR-04	Reliability	Return structured errors, verify DB health, preserve relational integrity, and log important changes.
NFR-05	Availability	Containers may restart automatically and health checks detect DB failure. SLA/uptime target: not specified.
NFR-06	Usability	Provide consistent navigation, filters, labels, feedback, status badges, loading states, confirmation prompts, and readable exports.
NFR-07	Maintainability	Separate UI, routes, controllers, services, models, middleware, types, and automated tests; document API with OpenAPI.
NFR-08	Scalability	Use indexed queries, pagination, shared DB rate-limit buckets, and stateless signed sessions; capacity target: not specified.
NFR-09	Compatibility	Support modern browsers, Node.js 20.19+/22.12+, and a MySQL-compatible DB; Docker is an optional deployment path.
NFR-10	Backup/Recovery	Allow administrators to create, list, download, and restore SQL backups; RPO/RTO and schedule are not specified.

9.1 Security Boundary

Frontend route protection improves usability but is not the trust boundary. The Express API verifies the session and required role for every protected operation. Production configuration must use HTTPS, a strong JWT secret, protected environment files, least-privilege database credentials, and persistent secured upload/backup storage.

10. Constraints, Testing, Conclusion, and Future Enhancements

10.1 Constraints and Acceptance Criteria

Technical constraints are the selected React/Express/MySQL stack, browser-based access, writable upload/backup storage, and availability of mysql utilities. Business rules for statutory assessment, invoices, payments, penalties, appeals, and public access are not specified in the current project.

- AC-01: active users sign in and see only role-permitted screens/actions; unauthorized API access is rejected.
- AC-02: valid property/client data can be listed, filtered, saved, linked, mapped, and exported according to role.
- AC-03: admin analytics, tax PDF, audit, user administration, and backup operations produce correct controlled results.

10.2 Recommended Testing

Retain the implemented Vitest component tests, Jest unit/integration tests, and Playwright end-to-end tests. Test RBAC denials as well as success paths; validation boundaries; CRUD and image workflows; reports/PDFs; map coordinates; settings; audit entries; backup restore in an isolated database; security headers, CSRF and rate limits; responsive layouts; and a production-like Docker deployment. Use a dedicated test database.

10.3 Known Limitations and Future Enhancements

Status	Item
Known limitation	The 1% tax is explicitly illustrative and based on listing price, not a formal assessment.
Known limitation	Forgot-password delivery, formal WCAG result, SLA, retention policy, and automated backup schedule are not specified.
Future Enhancement	Add governed assessments, configurable rates, exemptions, invoices, payments, receipts, arrears, penalties, appeals, and reconciliation.
Future Enhancement	Add citizen self-service, approved public information, stronger accessibility testing, scheduled off-site backups, and disaster-recovery targets.

10.4 Conclusion

The current system is a coherent, test-backed administrative property platform with strong role separation, reporting, mapping, audit, and recovery foundations. It supports an academic demonstration well, provided the illustrative tax output is not presented as a complete statutory tax-payment system.

10.5 Approval

Prepared by	Reviewed by	Approved by
Name: _____ Signature: _____ Date: _____	Name: _____ Signature: _____ Date: _____	Name: _____ Signature: _____ Date: _____